Import libraries

In [108…
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import sqlite3 as sqlite3
```

In [109…
```
#importing data base
con = sqlite3.connect("customer_churn.db")
sqlquery = """ SELECT name
             FROM sqlite_master
             WHERE type = 'table' """
tables = pd.read_sql(sqlquery,con)
for table_name in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table_name}",con)
    globals()[f"df_{table_name}"]=df
    print(f"dataframe created: {table_name}")
```

```
dataframe created: db_customer
dataframe created: db_subscription
dataframe created: db_support
```

Data cleaning

In [110…
```
df_db_customer.head(10)
```

Out[110…

	customerid	name	country	state	gender	dob	interests	pincode
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	None
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	NaN	None
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	None
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	NaN	None
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	None
5	0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10 00:00:00	NaN	None
6	0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00	NaN	None
7	0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00	NaN	None
8	0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07 00:00:00	NaN	None
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00	NaN	None

In [111…

```
df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   customerid  21 non-null     str
 1   name        21 non-null     str
 2   country     18 non-null     str
 3   state       21 non-null     str
 4   gender      21 non-null     str
 5   dob         21 non-null     str
 6   interests   4 non-null      str
 7   pincode     0 non-null      object
dtypes: object(1), str(7)
memory usage: 2.5+ KB
```

In [112…
```
# Removing columns (interests,pincode)
df_db_customer= df_db_customer.drop(['interests','pincode'] , axis = 1)
df_db_customer
```

Out[112…

	customerid	name	country	state	gender	dob
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00
5	0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10 00:00:00
6	0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00
7	0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00
8	0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07 00:00:00
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00
10	0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00
11	0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00
12	0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01 00:00:00
13	0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00
15	0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00

In [113…
```
# Filling null values
country_mapping = df_db_customer.dropna(subset='country').set_index('state')['co
df_db_customer['country'] = df_db_customer['country'].fillna(df_db_customer['sta
df_db_customer
```

Out[113...

	customerid	name	country	state	gender	dob
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00
5	0013-MHZWF	durga	India	Delhi	Women	1988-12-10 00:00:00
6	0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00
7	0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00
8	0015-UOCOJ	maya	Nepal	Kathmandu	Women	1985-07-07 00:00:00
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00
10	0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00
11	0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00
12	0018-NYROU	chitra	India	Telangana	Female	2004-12-01 00:00:00
13	0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00
15	0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00

In [114...

```
# renaming the name column
df_db_customer = df_db_customer.rename(columns = {'name':'customer_name'})
df_db_customer
```

Out[114...

	customerid	customer_name	country	state	gender	dob
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00
5	0013-MHZWF	durga	India	Delhi	Women	1988-12-10 00:00:00
6	0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00
7	0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00
8	0015-UOCOJ	maya	Nepal	Kathmandu	Women	1985-07-07 00:00:00
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00
10	0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00
11	0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00
12	0018-NYROU	chitra	India	Telangana	Female	2004-12-01 00:00:00
13	0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00
15	0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00

In [115…

```
# standardization of gender columns
df_db_customer['gender']= df_db_customer['gender'].replace({'Men':'Male','Women'
df_db_customer
```

Out[115...

	customerid	customer_name	country	state	gender	dob
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00
5	0013-MHZWF	durga	India	Delhi	Female	1988-12-10 00:00:00
6	0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00
7	0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00
8	0015-UOCOJ	maya	Nepal	Kathmandu	Female	1985-07-07 00:00:00
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00
10	0017-DINOC	shiva	India	Maharashtra	Male	1997-01-22 00:00:00
11	0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00
12	0018-NYROU	chitra	India	Telangana	Female	2004-12-01 00:00:00
13	0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Male	1979-11-11 00:00:00
15	0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Male	1991-05-14 00:00:00
20	0023-UYUPN	sudevi	India	Maharashtra	Female	1977-10-06 00:00:00

In [116…
```
# change datatype of dob column
df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])
df_db_customer
```

Out[116…

	customerid	customer_name	country	state	gender	dob
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05
5	0013-MHZWF	durga	India	Delhi	Female	1988-12-10
6	0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21
7	0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14
8	0015-UOCOJ	maya	Nepal	Kathmandu	Female	1985-07-07
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29
10	0017-DINOC	shiva	India	Maharashtra	Male	1997-01-22
11	0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18
12	0018-NYROU	chitra	India	Telangana	Female	2004-12-01
13	0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Male	1979-11-11
15	0020-INWCK	parvati	India	Delhi	Female	1986-02-28
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Male	1991-05-14
20	0023-UYUPN	sudevi	India	Maharashtra	Female	1977-10-06

In [117…
```
df_db_subscription
```

Out[117…

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	cor
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	
5	0013-MHZWF	2022-06-18	Paid	2025-06-18	Standard	
6	0013-SMEOE	2021-09-30	Refferal	2024-09-30	Basic	
7	0014-BMAQU	2020-02-14	Organic	2025-02-14	Premium	
8	0015-UOCOJ	2023-07-22	Organic	2024-07-22	Standard	
9	0016-QLJIS	2022-04-03	Organic	2025-04-03	Basic	
10	0017-DINOC	2021-12-01	Organic	2025-12-01	Premium	
11	0017-IUDMW	2023-03-17	Refferal	2024-03-17	Standard	
12	0018-NYROU	2020-10-09	Organic	2025-10-09	Standard	
13	0019-EFAEP	2022-08-25	Refferal	2024-08-25	Basic	
14	0019-GFNTW	2019-11-11	Paid	2025-11-11	Premium	
15	0020-INWCK	2023-05-06	Organic	2025-05-06	Standard	
16	0020-JDNXP	2022-01-19	Organic	2024-01-19	Premium	
17	0021-IKXGC	2021-07-07	Paid	2025-07-07	Standard	
18	0022-TCJCI	2023-09-14	Refferal	2024-09-14	Basic	
19	0023-HGHWL	2020-06-23	Organic	2025-06-23	Premium	
20	0023-UYUPN	2022-12-31	Paid	2025-12-31	Standard	

In [118…

```
df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   customerid               21 non-null     str
 1   subscription_start_date  21 non-null     str
 2   subscription_type        21 non-null     str
 3   renewal_date             21 non-null     str
 4   plan_type                21 non-null     str
 5   contract_type            21 non-null     str
 6   cancellation_date        6 non-null      str
 7   cancellation_reason      6 non-null      str
 8   monthly_charges          21 non-null     float64
 9   cltv                     21 non-null     int64
 10  churn_score              21 non-null     int64
dtypes: float64(1), int64(2), str(8)
memory usage: 3.1 KB
```

In [119…

```
# change datyatype of dates
date_col = ['subscription_start_date','renewal_date','cancellation_date']
df_db_subscription[date_col] = df_db_subscription[date_col].apply(pd.to_datetime
df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   customerid               21 non-null     str
 1   subscription_start_date  21 non-null     datetime64[us]
 2   subscription_type        21 non-null     str
 3   renewal_date             21 non-null     datetime64[us]
 4   plan_type                21 non-null     str
 5   contract_type            21 non-null     str
 6   cancellation_date        6 non-null      datetime64[us]
 7   cancellation_reason      6 non-null      str
 8   monthly_charges          21 non-null     float64
 9   cltv                     21 non-null     int64
 10  churn_score              21 non-null     int64
dtypes: datetime64[us](3), float64(1), int64(2), str(5)
memory usage: 2.7 KB
```

In [120…

```
df_db_support
```

Out[120…

	customerid	complaint_date	escalations	csat_score	col_1	comment
0	0003-MKNFE	2024-08-28 00:00:00	N	60	None	service issue
1	0003-MKNFE	2024-08-28 00:00:00	Y	10	None	demaned refund
2	0013-EXCHZ	2024-01-20 00:00:00	Y	20	None	NaN
3	0013-MHZWF	2025-03-18 00:00:00	N	90	None	guidance to renew
4	0013-SMEOE	2024-11-01 00:00:00	N	30	None	NaN
5	0017-IUDMW	2024-04-10 00:00:00	Y	25	None	NaN
6	0019-EFAEP	2024-09-27 00:00:00	Y	30	None	NaN
7	0022-TCJCI	2024-09-13 00:00:00	Y	10	None	NaN
8	0022-TCJCI	2024-09-14 00:00:00	N	90	None	received refund

In [121…

```
df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype 
---  ------          --------------  ----- 
 0   customerid      9 non-null      str   
 1   complaint_date  9 non-null      str   
 2   escalations     9 non-null      str   
 3   csat_score      9 non-null      int64 
 4   col_1           0 non-null      object
 5   comment         4 non-null      str   
dtypes: int64(1), object(1), str(4)
memory usage: 895.0+ bytes
```

In [122…

```
# Removing columns ( col1,comment)
df_db_support = df_db_support.drop(['col_1','comment'] , axis = 1)
df_db_support
```

Out[122...

	customerid	complaint_date	escalations	csat_score
0	0003-MKNFE	2024-08-28 00:00:00	N	60
1	0003-MKNFE	2024-08-28 00:00:00	Y	10
2	0013-EXCHZ	2024-01-20 00:00:00	Y	20
3	0013-MHZWF	2025-03-18 00:00:00	N	90
4	0013-SMEOE	2024-11-01 00:00:00	N	30
5	0017-IUDMW	2024-04-10 00:00:00	Y	25
6	0019-EFAEP	2024-09-27 00:00:00	Y	30
7	0022-TCJCI	2024-09-13 00:00:00	Y	10
8	0022-TCJCI	2024-09-14 00:00:00	N	90

In [123...

```
# Change datatype
df_db_support['complaint_date'] = pd.to_datetime(df_db_support['complaint_date']
df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      9 non-null      str
 1   complaint_date  9 non-null      datetime64[us]
 2   escalations     9 non-null      str
 3   csat_score      9 non-null      int64
dtypes: datetime64[us](1), int64(1), str(2)
memory usage: 519.0 bytes
```

feature engineering

In [124...

```
#creating new column by using existing column
df_db_subscription['churn_flag'] = np.where(df_db_subscription['cancellation_dat
df_db_subscription
```

Out[124...

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	cor
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	
5	0013-MHZWF	2022-06-18	Paid	2025-06-18	Standard	
6	0013-SMEOE	2021-09-30	Refferal	2024-09-30	Basic	
7	0014-BMAQU	2020-02-14	Organic	2025-02-14	Premium	
8	0015-UOCOJ	2023-07-22	Organic	2024-07-22	Standard	
9	0016-QLJIS	2022-04-03	Organic	2025-04-03	Basic	
10	0017-DINOC	2021-12-01	Organic	2025-12-01	Premium	
11	0017-IUDMW	2023-03-17	Refferal	2024-03-17	Standard	
12	0018-NYROU	2020-10-09	Organic	2025-10-09	Standard	
13	0019-EFAEP	2022-08-25	Refferal	2024-08-25	Basic	
14	0019-GFNTW	2019-11-11	Paid	2025-11-11	Premium	
15	0020-INWCK	2023-05-06	Organic	2025-05-06	Standard	
16	0020-JDNXP	2022-01-19	Organic	2024-01-19	Premium	
17	0021-IKXGC	2021-07-07	Paid	2025-07-07	Standard	
18	0022-TCJCI	2023-09-14	Refferal	2024-09-14	Basic	
19	0023-HGHWL	2020-06-23	Organic	2025-06-23	Premium	
20	0023-UYUPN	2022-12-31	Paid	2025-12-31	Standard	

In [125…

```
# merge dataframe before fixing tables
customer_subscription = pd.merge(df_db_customer , df_db_subscription , on = 'cus
df = pd.merge(customer_subscription , df_db_support, on = 'customerid' ,how = 'o
df
```

Out[125...

	customerid	customer_name	country	state	gender	dob	subscription_star
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12	2021
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23	2020
2	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23	2020
3	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15	2022
4	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30	2019
5	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05	2023
6	0013-MHZWF	durga	India	Delhi	Female	1988-12-10	2022
7	0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21	2021
8	0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14	2020
9	0015-UOCOJ	maya	Nepal	Kathmandu	Female	1985-07-07	2023
10	0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29	2022
11	0017-DINOC	shiva	India	Maharashtra	Male	1997-01-22	2021
12	0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18	2023
13	0018-NYROU	chitra	India	Telangana	Female	2004-12-01	2020
14	0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25	2022
15	0019-GFNTW	Madhav	India	Uttar Pradesh	Male	1979-11-11	2019
16	0020-INWCK	parvati	India	Delhi	Female	1986-02-28	2023
17	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19	2022
18	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02	2021
19	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30	2023
20	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30	2023

	customerid	customer_name	country	state	gender	dob	subscription_star
21	0023-HGHWL	rishabh	India	Uttar Pradesh	Male	1991-05-14	2020
22	0023-UYUPN	sudevi	India	Maharashtra	Female	1977-10-06	2022

In [126…
```
df.shape
```
Out[126… (23, 20)

In [127…
```
df_db_support.shape
```
Out[127… (9, 4)

In [128…
```
df_db_customer.shape
```
Out[128… (21, 6)

In [129…
```
df_db_subscription.shape
```
Out[129… (21, 12)

In [130…
```
df_db_subscription['customerid'].nunique()
```
Out[130… 21

In [131…
```
df_db_customer['customerid'].nunique()
```
Out[131… 21

In [132…
```
df_db_support['customerid'].nunique()
```
Out[132… 7

In [133…
```
df_db_support
```

Out[133...

	customerid	complaint_date	escalations	csat_score
0	0003-MKNFE	2024-08-28	N	60
1	0003-MKNFE	2024-08-28	Y	10
2	0013-EXCHZ	2024-01-20	Y	20
3	0013-MHZWF	2025-03-18	N	90
4	0013-SMEOE	2024-11-01	N	30
5	0017-IUDMW	2024-04-10	Y	25
6	0019-EFAEP	2024-09-27	Y	30
7	0022-TCJCI	2024-09-13	Y	10
8	0022-TCJCI	2024-09-14	N	90

In [134...

```
# creating new column by using groupby
df_db_support['complaint_count'] =  df_db_support.groupby('customerid')['custome
df_db_support
```

Out[134...

	customerid	complaint_date	escalations	csat_score	complaint_count
0	0003-MKNFE	2024-08-28	N	60	2
1	0003-MKNFE	2024-08-28	Y	10	2
2	0013-EXCHZ	2024-01-20	Y	20	1
3	0013-MHZWF	2025-03-18	N	90	1
4	0013-SMEOE	2024-11-01	N	30	1
5	0017-IUDMW	2024-04-10	Y	25	1
6	0019-EFAEP	2024-09-27	Y	30	1
7	0022-TCJCI	2024-09-13	Y	10	2
8	0022-TCJCI	2024-09-14	N	90	2

In [135...

```
# sorting datframe on basis of date
df_db_support = df_db_support.sort_values('complaint_date').drop_duplicates('cus
df_db_support
```

Out[135...

	customerid	complaint_date	escalations	csat_score	complaint_count
2	0013-EXCHZ	2024-01-20	Y	20	1
5	0017-IUDMW	2024-04-10	Y	25	1
1	0003-MKNFE	2024-08-28	Y	10	2
8	0022-TCJCI	2024-09-14	N	90	2
6	0019-EFAEP	2024-09-27	Y	30	1
4	0013-SMEOE	2024-11-01	N	30	1
3	0013-MHZWF	2025-03-18	N	90	1

In [136...

```
# sorting data frame
df_db_support= df_db_support.sort_values('customerid')
df_db_support
```

Out[136...

	customerid	complaint_date	escalations	csat_score	complaint_count
1	0003-MKNFE	2024-08-28	Y	10	2
2	0013-EXCHZ	2024-01-20	Y	20	1
3	0013-MHZWF	2025-03-18	N	90	1
4	0013-SMEOE	2024-11-01	N	30	1
5	0017-IUDMW	2024-04-10	Y	25	1
6	0019-EFAEP	2024-09-27	Y	30	1
8	0022-TCJCI	2024-09-14	N	90	2

In [137...

```
#merge dataframe after fixing tables
df = (df_db_customer.merge(df_db_subscription , on = 'customerid',how = 'left')
                    .merge(df_db_support , on = 'customerid',how = 'left'))
df
```

Out[137...

	customerid	customer_name	country	state	gender	dob	subscription_star
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12	2021
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23	2020
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15	2022
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30	2019
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05	2023
5	0013-MHZWF	durga	India	Delhi	Female	1988-12-10	2022
6	0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21	2021
7	0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14	2020
8	0015-UOCOJ	maya	Nepal	Kathmandu	Female	1985-07-07	2023
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29	2022
10	0017-DINOC	shiva	India	Maharashtra	Male	1997-01-22	2021
11	0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18	2023
12	0018-NYROU	chitra	India	Telangana	Female	2004-12-01	2020
13	0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25	2022
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Male	1979-11-11	2019
15	0020-INWCK	parvati	India	Delhi	Female	1986-02-28	2023
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19	2022
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02	2021
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30	2023
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Male	1991-05-14	2020
20	0023-UYUPN	sudevi	India	Maharashtra	Female	1977-10-06	2022

21 rows × 21 columns

Data analysis

In [138…
```
df.to_csv('export_churn_data.csv', index= False)
```

In [139…
```
df.columns
```

Out[139…
```
Index(['customerid', 'customer_name', 'country', 'state', 'gender', 'dob',
       'subscription_start_date', 'subscription_type', 'renewal_date',
       'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'complaint_date', 'escalations', 'csat_score',
       'complaint_count'],
      dtype='str')
```

In [140…
```
# churn rate
churn_value = df['churn_flag'].mean()*100
print("churn rate is", round(churn_value,2) ,"%" )
```

```
churn rate is 28.57 %
```

In [141…
```
# Retention rate
retention_value = 100 - churn_value
print("churn rate is", round(retention_value,2) ,"%" )
```

```
churn rate is 71.43 %
```

In [142…
```
# churn according to subscription type
subscribe = df.groupby('subscription_type').agg(
                                        churn_rate = ('churn_flag','mean'),
                                        monthly_revenue= ('monthly_charges','su
                                        customer_count = ('customerid','count')
subscribe['churn_rate'] = subscribe['churn_rate']*100
subscribe
```

Out[142…

	subscription_type	churn_rate	monthly_revenue	customer_count
0	Organic	0.000000	145.91	9
1	Paid	16.666667	174.94	6
2	Refferal	83.333333	74.94	6

In [143…
```
# finding churn by state
state= df.groupby('state').agg(
                        churn_rate= ('churn_flag','mean'),
                        monthly_revenue= ('monthly_charges','sum'),
                        customer_count = ('customerid','count')).reset_index()
state['churn_rate'] =state['churn_rate']*100
state
```

Out[143...

	state	churn_rate	monthly_revenue	customer_count
0	Delhi	25.000000	52.96	4
1	Karnataka	100.000000	20.98	2
2	Kathmandu	0.000000	20.98	2
3	Maharashtra	0.000000	50.97	3
4	Meghalaya	66.666667	42.97	3
5	Nagaland	0.000000	22.99	1
6	Rajasthan	0.000000	36.98	2
7	Telangana	50.000000	30.98	2
8	Uttar Pradesh	0.000000	115.98	2

In [144...
```
# finding churn by plan
plan_churn = df.groupby('plan_type')['churn_flag'].sum()
plan_churn
```

Out[144...
```
plan_type
Basic       3
Premium     1
Standard    2
Name: churn_flag, dtype: int64
```

In [145...
```
#avg revenue per user
apru = df['monthly_charges'].mean()
print("APRU is",apru.round(2))
```

```
APRU is 18.85
```

In [146...
```
df.columns
```

Out[146...
```
Index(['customerid', 'customer_name', 'country', 'state', 'gender', 'dob',
       'subscription_start_date', 'subscription_type', 'renewal_date',
       'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'complaint_date', 'escalations', 'csat_score',
       'complaint_count'],
      dtype='str')
```

In [147...
```
# Tenure days :how many days a person used our service:
today = pd.Timestamp.today()
df['tenture days'] = np.where(df['cancellation_date'].notna(),
                              (df['cancellation_date'] - df['subscription_start_
                              (today - df['subscription_start_date']).dt.days
                             )

tenture=df['tenture days'].mean()
print("Average tenture days is",tenture)
```

```
Average tenture days is 1571.5714285714287
```

In [148...
```
# revenue at risk
revenue_risk = df.loc[df['churn_flag'] == 1,'monthly_charges'].sum()
print('revenue at risk (Rs in k)',revenue_risk)
```

```
revenue at risk (Rs in k) 73.94
```

In [149…
```
#Escalation rate
esc_rate= (df['escalations'] == 'Y').mean()*100
print("escalation rate is" , round(esc_rate,2),"%")
```

```
escalation rate is 19.05 %
```

In [150…
```
# avg complaint rate per user
avg_complaint = df['complaint_count'].sum() / df['customerid'].nunique()*100
print("Average complaint rate per user",round(avg_complaint ,2),"%")
```

```
Average complaint rate per user 42.86 %
```

In [151…
```
# correlation b/w escalation & churn
df['escalations'] = np.where(df['escalations']=='Y',1,0)
corr = df[['escalations' , 'churn_flag']]
correlation = corr['escalations'].corr(df['churn_flag'])*100
print('correlation b/w escalation & churn is',round(correlation,2),'%')
```

```
correlation b/w escalation & churn is 76.7 %
```

In [152…
```
# finding the level of churn risk
conditions = [
             (df['churn_score']<50),
             (df['churn_score']>=50) & (df['churn_score']<70),
             (df['churn_score']>=70) & (df['churn_score']<=99)
             ]
options = ['low','med','high']
df['churn_risk']=np.select(conditions,options,default= "unknown")

df[['churn_score','churn_risk']]
```

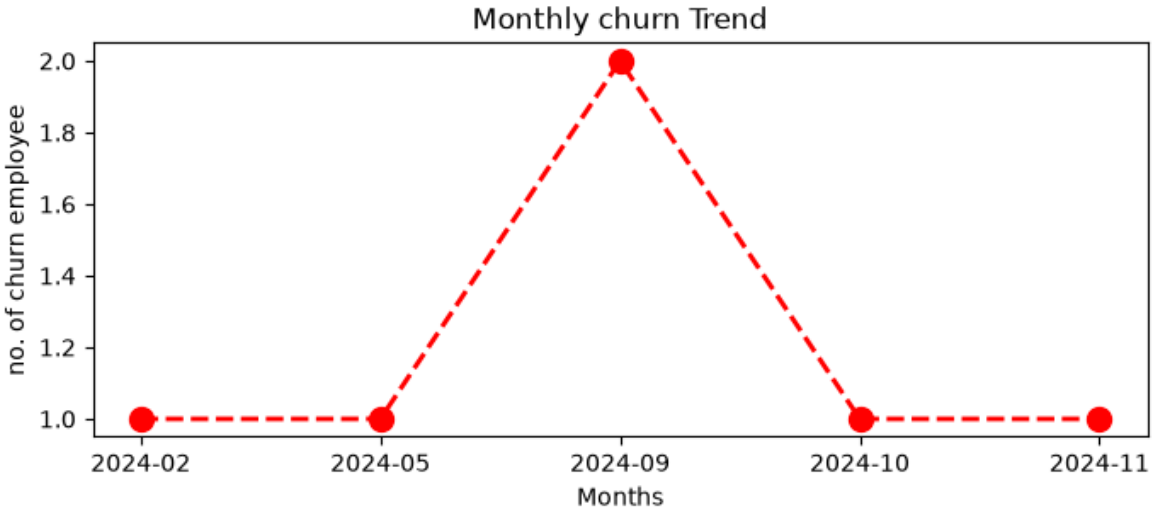
Out[152...

	churn_score	churn_risk
0	12	low
1	91	high
2	34	low
3	8	low
4	88	high
5	22	low
6	79	high
7	5	low
8	34	low
9	41	low
10	14	low
11	83	high
12	19	low
13	76	high
14	3	low
15	58	med
16	62	med
17	27	low
18	99	high
19	7	low
20	47	low

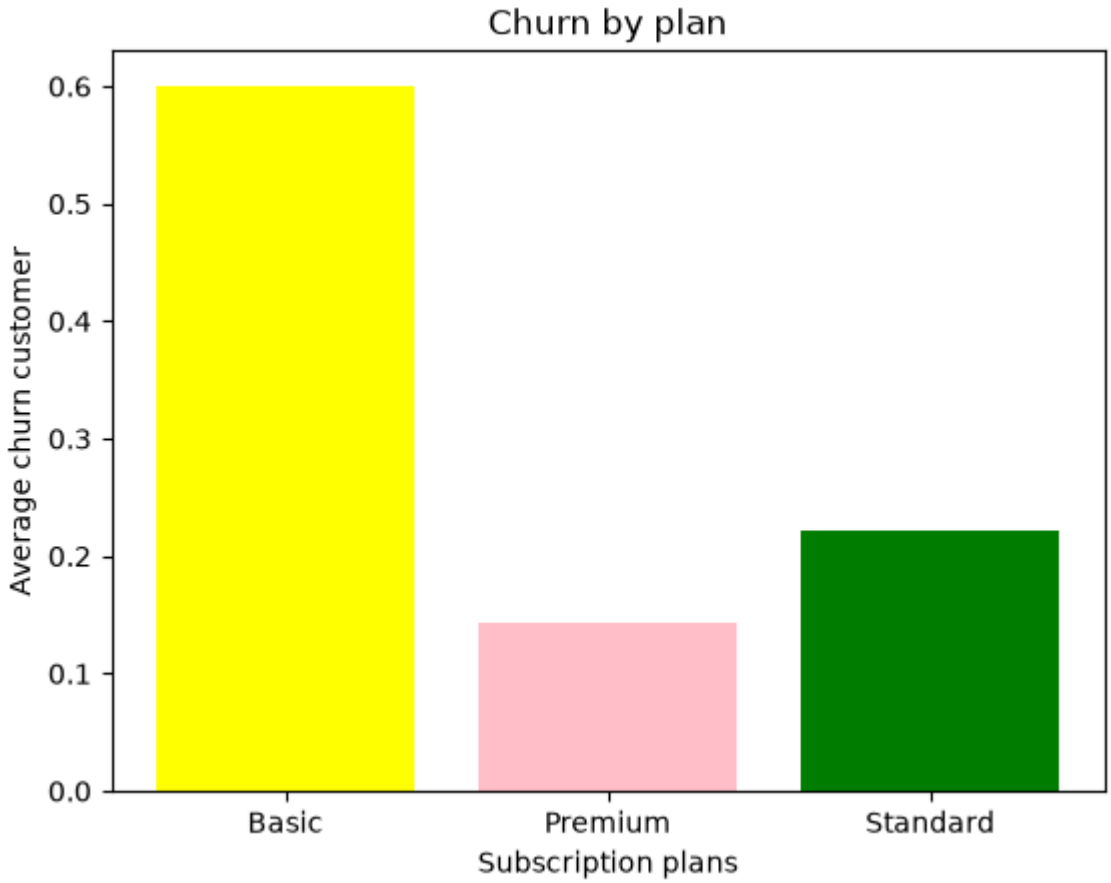
DATA VISUALISATION BY USING MATPLOTLIB

In [153...
```
df_visual = df.copy()
```

In [154...
```
#Monthly churn trend
df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M
churn_trend = df_visual[df_visual['churn_flag']==1].groupby('cancellation_month'
plt.figure (figsize=(8,3))
plt.xlabel("Months")
plt.ylabel("no. of churn employee")
plt.title("Monthly churn Trend")
plt.plot(churn_trend.index.astype('str'),churn_trend.values, color= 'red',linest
plt.show()
```



In [155…
```
#churn by plan type
churn_plan = df_visual['churn_flag'].groupby(df_visual['plan_type']).mean()
colours = ['yellow','pink','green']
plt.bar(churn_plan.index,churn_plan.values,color= colours)
plt.xlabel("Subscription plans")
plt.ylabel("Average churn customer")
plt.title("Churn by plan")
plt.figure(figsize=(7,4))
plt.show()
```



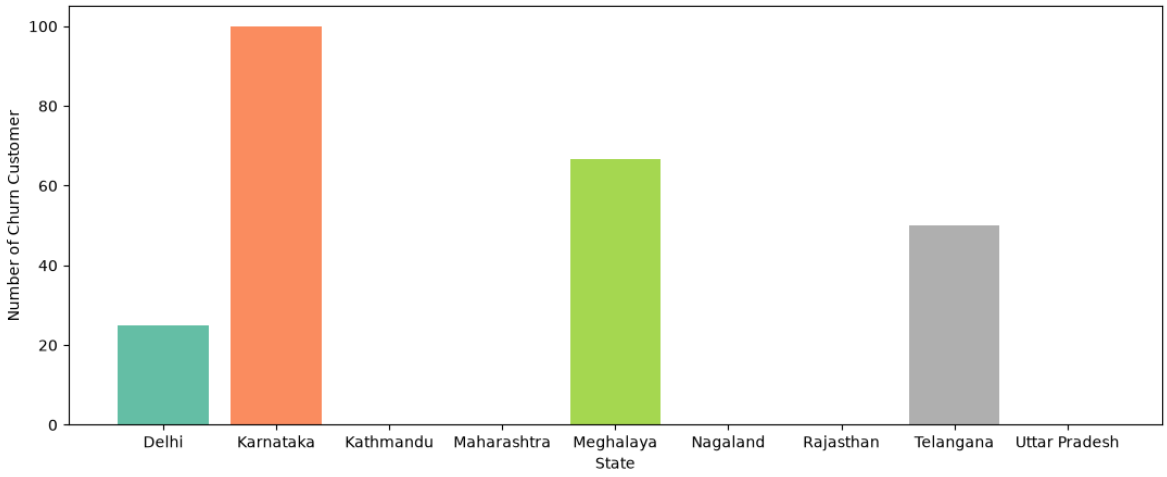

```
<Figure size 700x400 with 0 Axes>
```

In [157…
```
# finding churn statewise
state_churn=df_visual['churn_flag'].groupby(df_visual['state']).mean()*100
round(state_churn,2)
plt.figure(figsize=(13,5))
```

```
colours = plt.cm.Set2(np.linspace (0 ,1,len(state_churn)))
plt.bar(state_churn.index , state_churn.values,color= colours)
plt.xlabel("State")
plt.ylabel("Number of Churn Customer")
```

Out[157…

```
Text(0, 0.5, 'Number of Churn Customer')
```



Data Visualisation using Seaborn

In [158…

```
df_visual.columns
```

Out[158…

```
Index(['customerid', 'customer_name', 'country', 'state', 'gender', 'dob',
       'subscription_start_date', 'subscription_type', 'renewal_date',
       'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'complaint_date', 'escalations', 'csat_score',
       'complaint_count', 'tenture days', 'churn_risk', 'cancellation_month'],
      dtype='str')
```

In [159…

```
df_encoded=df_visual[['subscription_type','plan_type','contract_type','churn_sco
df_encoded.head()
```

Out[159…

	subscription_type	plan_type	contract_type	churn_score	churn_flag	escalations	chu
0	Refferal	Standard	Annual	12	0	0	
1	Paid	Premium	Annual	91	1	1	
2	Organic	Basic	Monthly	34	0	0	
3	Paid	Premium	Annual	8	0	0	
4	Refferal	Standard	Monthly	88	1	1	

In [160…

```
#encoding str values into int#
order={'subscription_type':['Organic','Refferal', 'Paid'],
           'plan_type':['Basic','Standard', 'Premium'],
           'contract_type':['Monthly','Annual'],
           'churn_risk':['low','med','high']}
for key,col in order.items():
    df_encoded[key]=pd.Categorical(df_encoded[key].astype('category'),categories
```

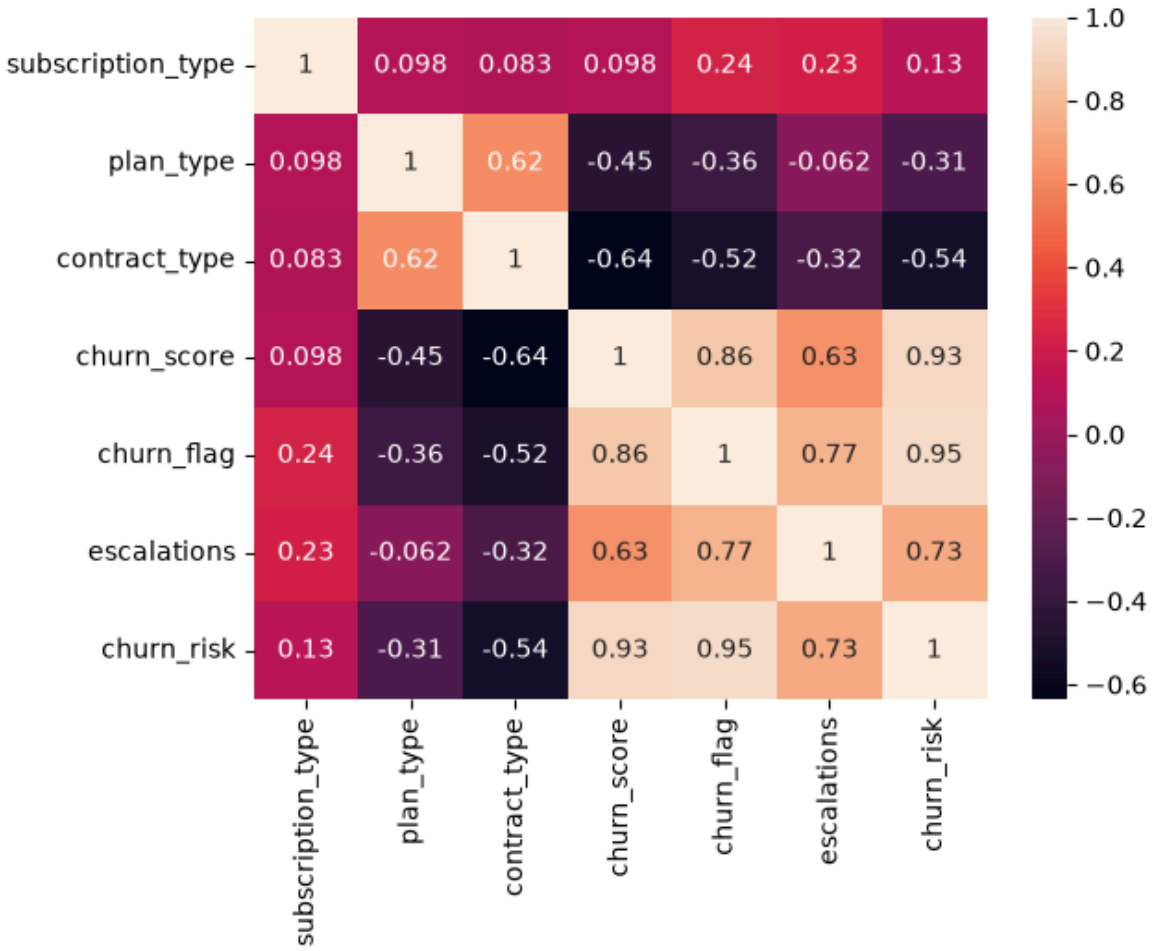
In [161…
```
#encode data to plot heatmap
df_encoded = df_encoded[['subscription_type','plan_type', 'contract_type','churn
df_encoded.head()
```

Out[161…

	subscription_type	plan_type	contract_type	churn_score	churn_flag	escalations	chu
0	1	1	1	12	0	0	
1	2	2	1	91	1	1	
2	0	0	0	34	0	0	
3	2	2	1	8	0	0	
4	1	1	0	88	1	1	

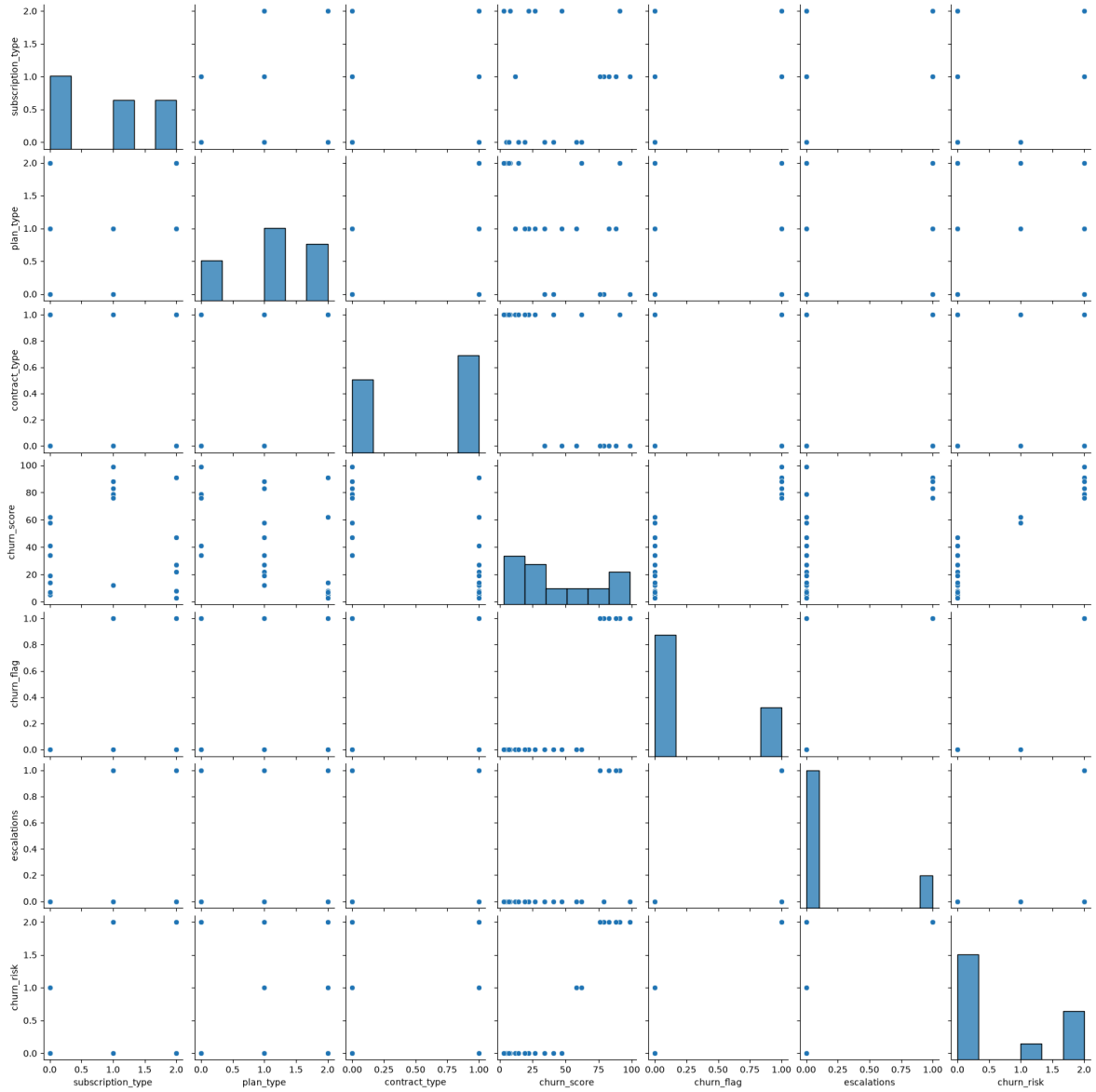
In [162…
```
sns.heatmap(df_encoded.corr(),annot=True)
```

Out[162… <Axes: >



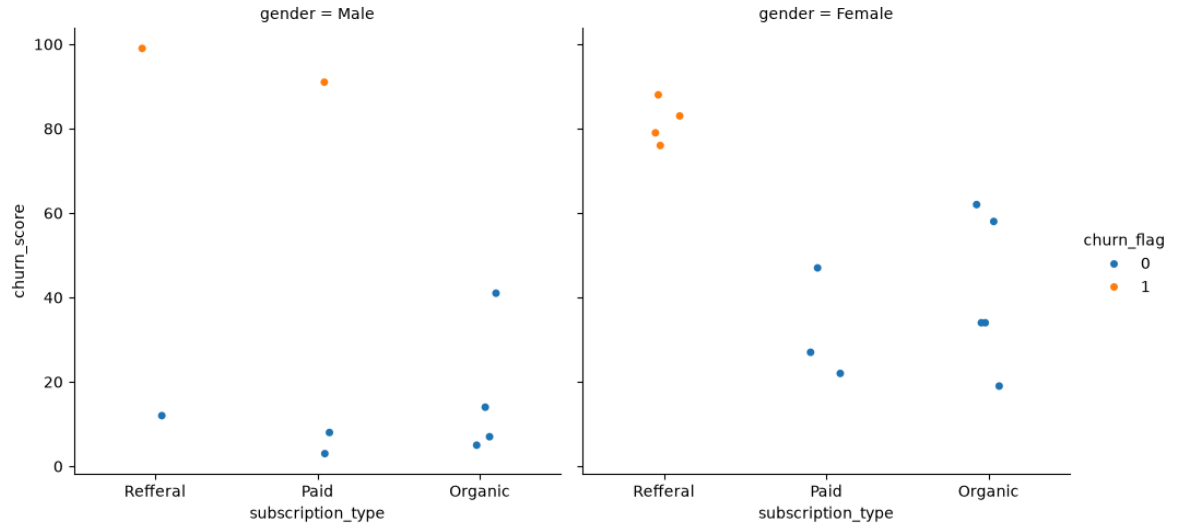
In [163…
```
sns.pairplot(df_encoded)
```

Out[163… <seaborn.axisgrid.PairGrid at 0x21695360690>



In [165…
```
sns.catplot(data=df_visual,x='subscription_type',y='churn_score',hue='churn_flag
```

Out[165…
```
<seaborn.axisgrid.FacetGrid at 0x216984b5d10>
```



SQL with Python and Pandas

In [176…
```
# creating and connecting a new table
conn = sqlite3.connect("user_database.sqlite")
```

```
conn.execute("CREATE TABLE data_table(NAME VARCHAR[20],COUNTRY VARCHAR[30],BUDGE
conn.commit()
print("DATABASE CREATED")
```

```
---------------------------------------------------------------------------
OperationalError                          Traceback (most recent call last)
Cell In[176], line 3
      1 # creating and connecting a new table
      2 conn = sqlite3.connect("user_database.sqlite")
----> 3 conn.execute("CREATE TABLE data_table(NAME VARCHAR[20],COUNTRY VARCHAR[3
0],BUDGET INT ,INTEREST FLOAT)")
      4 conn.commit()
      5 print("DATABASE CREATED")

OperationalError: table data_table already exists
```

In [173…

```
# Inserting data into tables
cursor = conn.cursor()
cursor.execute( """INSERT INTO data_table VALUES
                ( "Rohit yadav" ,"Russia" , 5000 ,2.3),
                ("Aarti Bhardwaj" , "India" , 8000 , 5.2),
                ("Kajal Tripathi","India",7800,4),
                ("Shikhar Alhawat" ,"America",3000,0.9)
            """)
conn.commit()
print("DATA ENTER SUCCESSFULLY")
```

```
DATA ENTER SUCCESSFULLY
```

In [174…

```
# Displaying data
conn = sqlite3.connect("user_database.sqlite")
query = """ SELECT * FROM data_table"""
user_table = pd.read_sql(query ,conn)
user_table
```

Out[174…

	NAME	COUNTRY	BUDGET	INTEREST
0	Rohit yadav	Russia	5000	2.3
1	Aarti Bhardwaj	India	8000	5.2
2	Kajal Tripathi	India	7800	4.0
3	Shikhar Alhawat	America	3000	0.9

In [175…

```
# Aggeration on data
QUERY = """ SELECT COUNTRY ,sum(BUDGET) AS TOTAL_BUDGET
            FROM data_table
            GROUP BY COUNTRY """
user_data = pd.read_sql(QUERY , conn)
user_data
```

Out[175…

	COUNTRY	TOTAL_BUDGET
0	America	3000
1	India	15800
2	Russia	5000